

Backend Build PRD — Indigent

Post-shortlist build phase · Owners: Backend Dev A + Backend Dev B

ML Engineer owns model/RAG/CV internals, referenced at interface level only

Status: v1.1 — build specification

0. Changes Since Planning

1. Groq is the sanctioned testing path. Development/testing can run through the Groq API; shipped/demo configuration uses local models.
 2. Frontend integration status is unverified against the frozen API contract.
 3. `INFERENCE_MODE = local | groq` is explicit and global. It is never inferred or mixed per request.
 4. Sovereignty Monitor reflects the true active mode. Groq mode must visibly report external inference and must never display `AIR-GAPPED`.
 5. Default mode everywhere is `local`.
 6. Groq is opt-in only and is never a fallback when local inference fails.
 7. Demo/submission configuration ships with local mode enabled.
 8. **Policy Validation Layer** is inserted between the Agent Orchestrator and executable tools/sandbox.
 9. **Resource Manager** tracks model residency and basic VRAM/RAM/disk/concurrency constraints.
 10. **Artifact Validation** validates generated artifacts after creation and before approval.
 11. **Evidence Provenance** gives retrieved evidence stable document/chunk/source identifiers that flow into verification and artifacts.
 12. **Sovereignty state** explicitly separates inference mode from network policy and exposes connection/byte counters.
 13. **Security Test Matrix** becomes a build and integration-test requirement.
 14. **Security invariant**: LLM output MUST NEVER directly authorize an action.
-

1. Scope

This PRD defines the backend architecture, module boundaries, data contracts, engineering decisions, security controls, and acceptance requirements for the two backend engineers.

In scope

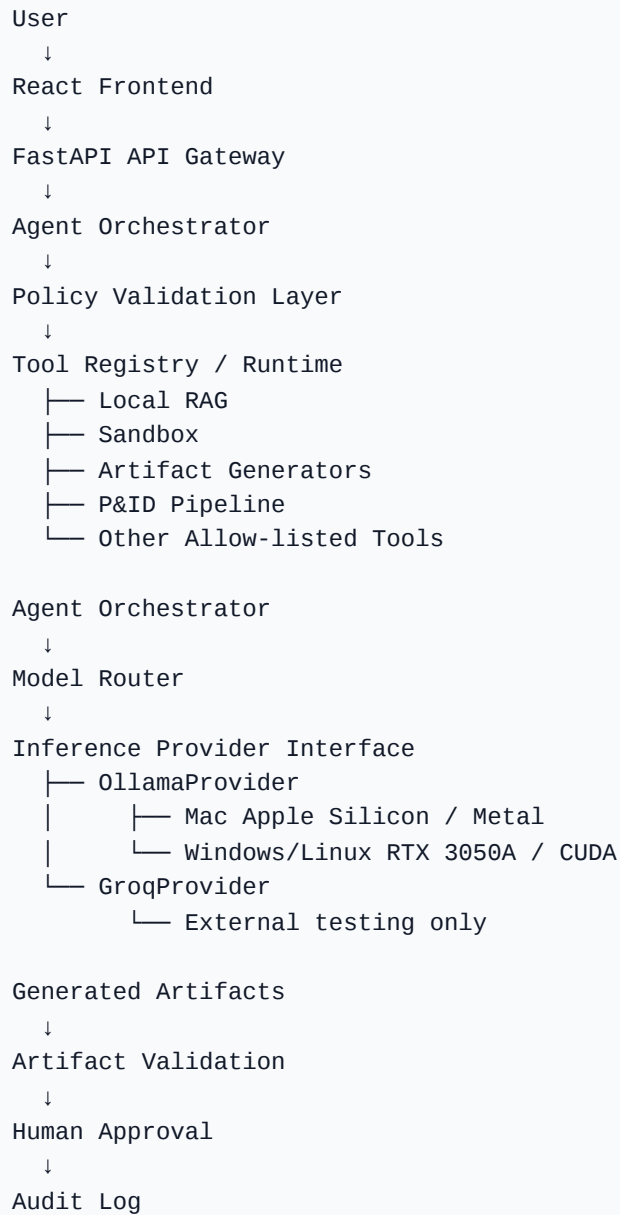
- FastAPI API gateway
- Inference provider abstraction
- Model registry and router
- Agent orchestration
- Policy validation/control plane

- Tool registry and dispatch
- Local RAG interfaces
- Sandbox execution
- Resource management
- Artifact generation and validation
- Evidence provenance
- P&ID pipeline interface
- Sovereignty monitoring
- Audit logging
- Security and failure testing

Out of scope

- Model training/fine-tuning
 - Model internals and prompting strategy
 - Frontend implementation
 - Full enterprise IAM
 - Multi-node orchestration
 - Production-grade multi-tenancy
 - Full CAD/P&ID reconstruction
 - Large-scale HA infrastructure
-

2. Architecture



Control-plane principle

The LLM proposes actions. It does not authorize actions.

```
LLM output
↓
Policy Validation
↓
Tool authorization
↓
Argument validation
↓
Resource checks
↓
Execution
```

Security invariant:

LLM output MUST NEVER directly authorize an action.

3. Inference Provider Interface

All inference providers implement a common interface.

```
class InferenceProvider(Protocol):
    def generate(
        self,
        model_id: str,
        messages: list[Message],
        tools: list[ToolSchema] | None
    ) -> InferenceResult: ...

    def is_local(self) -> bool: ...

    def health_check(self) -> ProviderHealth: ...
```

Implementations

- `OllamaProvider`
- `GroqProvider`

Provider SDKs MUST NOT be imported outside `providers/`.

The router, orchestrator, and tools must never directly call Ollama or Groq SDKs.

4. Global Inference Mode

The system has exactly one active inference mode:

```
INFERENCE_MODE=local
INFERENCE_MODE=groq
```

Local

- All inference stays on the local machine.
- Network policy is deny-all for external traffic.
- Submission/demo configuration uses this mode.

Groq

- External inference is explicitly active.
- Groq is allowed only for development/testing.
- Sovereignty Monitor MUST show external inference.
- It MUST NOT report `AIR-GAPPED`.
- External call counters must increment for every Groq request.
- Groq is never silently selected as a fallback.

Failure rule

```
Local inference unavailable
      ↓
FAIL LOUDLY
      ↓
DO NOT switch to Groq
```

5. Model Registry

The registry stores model capabilities independently from provider implementation.

Example:

```
{
  "id": "general-reasoning",
  "type": "reasoning",
  "modes": {
    "local": {
      "provider": "ollama",
      "profiles": {
        "mac_silicon": {
          "model_name": "qwen2.5:7b",
          "backend": "metal"
        },
        "rtx_3050a_4gb": {
          "model_name": "qwen2.5:3b",
          "backend": "cuda"
        }
      }
    },
    "groq": {
      "provider": "groq",
      "model_name": "<confirmed Groq model tag>"
    }
  },
  "tasks": ["planning", "analysis", "drafting"]
}
```

`LOCAL_HARDWARE_PROFILE` selects the local hardware profile.

6. Resource Manager

The Resource Manager provides a small, explicit resource-control layer.

Responsibilities

- Track VRAM availability
- Track RAM availability
- Track disk availability
- Track resident models
- Track model load/unload state
- Limit model concurrency
- Provide resource availability to the model router
- Fail cleanly when resource requirements cannot be satisfied

Example:

```
{  
  "model": "qwen2.5:7b",  
  "resident": true,  
  "memory_estimate_mb": 5200,  
  "available": true  
}
```

It is not a cluster scheduler. Its purpose is to prevent uncontrolled model loading and make resource-aware routing observable.

7. Model Router

Routing logic:

```
Rule-based fast path  
  ↓  
Capability check  
  ↓  
Resource/availability check  
  ↓  
Select model
```

If the active-mode model is unavailable, return a clean error.

The router MUST NOT silently switch inference modes.

8. Agent Orchestrator

The agent is implemented as a bounded state machine:

```
INTAKE
↓
CLASSIFY
↓
PLAN
↓
RETRIEVE
↓
TOOL
↓
VERIFY
↓
REPAIR
↓
ARTIFACT
↓
ARTIFACT_VALIDATE
↓
APPROVAL
↓
COMPLETE
```

Dynamic planning

PLAN must be genuinely dynamic. The reasoning model receives the task description, available tool schemas, and relevant constraints, then returns an ordered plan.

Sequencing must be determined by the generated plan, not hardcoded task-specific **if/else** logic.

Verification

For claims based on retrieved documents:

```
Claim + cited evidence chunk
↓
Cheap verification model
↓
boolean / confidence
```

Low confidence causes **VERIFY** to fail and enters **REPAIR**.

Bounds

```
MAX_REPAIR_ATTEMPTS = 3
PER_TOOL_TIMEOUT = 30 seconds
MODEL_GENERATION_TIMEOUT = 60 seconds
HARD_TASK_TIMEOUT = 5 minutes
```

9. Task State

```
{
  "task_id": "uuid",
  "user_request": "string",
  "task_type": "string",
  "plan": ["step1", "step2"],
  "current_state": "RETRIEVE",
  "inference_mode": "local | groq",
  "model_used": "general-reasoning",
  "tool_calls": [],
  "retrieved_chunks": [],
  "artifacts": [],
  "pid_graph": null,
  "errors": [],
  "verification_status": "pending | passed | failed",
  "final_result": null,
  "requires_human_approval": true,
  "approved_by": null
}
```

10. Policy Validation Layer

The Policy Validation Layer sits between the Agent Orchestrator and executable tools.

```
Agent Orchestrator
  ↓
Policy Validation Layer
  ↓
Tool Registry
  ↓
Tool Runtime / Sandbox
```

It is a security control plane, not an LLM feature.

Responsibilities

Before any tool execution:

1. Validate tool identity.
2. Check task-level authorization.
3. Validate all model-generated arguments.
4. Validate filesystem paths.
5. Enforce workspace boundaries.
6. Enforce allowed file types.

7. Enforce file-size limits.
8. Enforce resource limits.
9. Check whether human approval is required.
10. Emit an audit event.
11. Reject invalid requests before execution.

All model-generated tool names, arguments, paths, commands, filenames, and output locations are untrusted until validated.

Example:

```
../../../../etc/passwd
```

must be rejected before the underlying tool sees it.

11. Tool Registry

Allow-listed tools are enforced centrally.

```
ALLOWED_TOOLS = {
  "inspection": {
    "read_file",
    "ocr_document",
    "search_knowledge_base",
    "retrieve_section",
    "create_docx",
    "create_xlsx"
  },
  "coding": {
    "read_file",
    "write_file",
    "create_code",
    "execute_code",
    "run_tests"
  },
  "pid_analysis": {
    "analyze_image",
    "extract_pid_graph"
  }
}
```

Dispatch flow:

```
Agent plan
  ↓
Policy validation
  ↓
Task/tool authorization
  ↓
Argument validation
  ↓
Resource validation
  ↓
Tool dispatch
```

Unauthorized tools raise `ToolNotAllowedError` and generate an audit event.

12. Sandbox

Generated code executes in an isolated sandbox.

Minimum requirements:

- No network access
- Resource limits
- Ephemeral workspace
- Restricted filesystem
- No host filesystem access
- Timeout
- Process/output limits
- Audit events

The sandbox uses:

```
--network=none
```

in both local and Groq modes.

13. Evidence Provenance

Every retrieved evidence item must have stable provenance.

Example:

```
{
  "chunk_id": "doc_17_chunk_042",
  "document_id": "inspection_2026_09_12",
  "page": 7,
  "section": "Pressure Test",
  "source_hash": "sha256:...",
  "text": "..."
}
```

Provenance must survive the pipeline:

```
Document
  ↓
Chunk ID
  ↓
Claim
  ↓
Verification
  ↓
Artifact citation
```

14. Local RAG

Backend responsibilities:

- Receive knowledge files
- Store task/document metadata
- Invoke ML-owned ingestion interfaces
- Store/retrieve chunks
- Return provenance metadata
- Return citations
- Preserve document/chunk IDs

Storage:

```
Qdrant
  ↓
single local container
```

Task/workspace files remain on the local filesystem.

15. Artifact Generation and Validation

Artifacts:

- DOCX
- PPTX
- XLSX
- Graph JSON
- Code packages

Generation is followed by validation:

```
ARTIFACT
  ↓
ARTIFACT_VALIDATE
  ↓
APPROVAL
```

Validation

General:

- File exists
- File is readable
- File opens successfully
- Required metadata exists
- Artifact hash generated

Evidence-backed documents:

- Required sections exist
- Citations resolve
- Referenced evidence exists
- Claims have corresponding provenance

Calculations:

- Required calculation fields exist
- Deterministic calculator results are consistent
- Assumptions are recorded

XLSX:

- Required sheets exist
- Required cells/formulas exist
- Workbook opens successfully

Code:

- Source files exist
- Syntax validation succeeds
- Tests execute
- Exit status is recorded
- Resource-limit events are recorded

A structurally valid file is not automatically a semantically valid artifact.

16. P&ID Pipeline

ML owns P&ID internals.

Backend interface:

```
def extract_pid_graph(image_path: str) -> PIDGraph:  
    ...
```

Returns:

```
nodes  
edges  
overlay_image_path  
narrative  
confidence_summary
```

The backend treats the pipeline as one interface regardless of internal implementation.

17. Sovereignty Monitor

The Sovereignty Monitor reads `INFERENCE_MODE` directly. It must never infer mode from network observations or model names.

Local mode

```
{  
  "inference_mode": "local",  
  "status": "AIR-GAPPED",  
  "network_policy": "deny_all",  
  "external_api_calls": 0,  
  "external_connections": 0,  
  "external_bytes_out": 0,  
  "external_bytes_in": 0  
}
```

Expected:

```
External API Calls: 0
Internet Access: BLOCKED
```

Groq mode

```
{
  "inference_mode": "groq",
  "status": "EXTERNAL INFERENCE ACTIVE",
  "provider": "Groq",
  "network_policy": "groq_endpoint_only",
  "external_api_calls": 12,
  "external_connections": 1
}
```

Expected:

```
External API Calls: increments per Groq call
Internet Access: ALLOWED (Groq endpoint only)
```

Groq mode must NEVER display `AIR-GAPPED` .

The Groq API key is stored in environment variables and is never logged.

18. Network Enforcement

Local mode uses defense in depth:

```
Application
  ↓
Container/process policy
  ↓
OS/network firewall
  ↓
Packet/connection telemetry
```

The system should expose evidence of denied external connections, zero external API calls, and zero external bytes sent/received.

The sandbox remains network-disabled in every mode.

19. API Contract

Frontend integration is currently unverified.

The first backend task is to run the frontend against a stub backend implementing the expected contract and record mismatches.

Endpoints:

```
POST /api/chat
GET /api/tasks/{task_id}
GET /api/tasks/{task_id}/timeline
POST /api/tasks/{task_id}/approve

POST /api/files/upload
POST /api/knowledge/upload

GET /api/artifacts/{artifact_id}
GET /api/models

GET /api/monitoring/sovereignty

POST /api/pid/analyze
```

GET /api/tasks/{task_id} MUST include `inference_mode` .

GET /api/models should expose registry entries, resident model, active inference mode, availability, and relevant resource status.

POST /api/chat returns a task ID and SSE progress stream.

SSE event shape must be frozen with the frontend owner before integration is complete.

20. Storage

```
SQLite
├─ task state
└─ audit log

Qdrant
└─ vectors / retrieval metadata

Filesystem
└─ per-task workspaces
```

Audit entries MUST include `inference_mode` .

Artifact manifests should include:

```
artifact_id
task_id
artifact_type
created_at
artifact_hash
source_evidence_ids
verification_status
```

21. Audit Log

Minimum event categories:

```
TASK_CREATED
MODEL_SELECTED
PROVIDER_CALL
RETRIEVAL
TOOL_PROPOSED
POLICY_ALLOWED
POLICY_DENIED
TOOL_EXECUTED
SANDBOX_STARTED
SANDBOX_BLOCKED_NETWORK
VERIFICATION_STARTED
VERIFICATION_FAILED
REPAIR_STARTED
ARTIFACT_CREATED
ARTIFACT_VALIDATION_FAILED
ARTIFACT_VALIDATED
APPROVAL_REQUESTED
APPROVED
TASK_COMPLETED
TASK_FAILED
```

Relevant events include:

```
task_id
timestamp
inference_mode
component
action
status
error/details
```

Secrets MUST NOT be logged.

22. Security Invariants

1. LLM output MUST NEVER directly authorize an action.

2. Tool execution MUST pass through policy validation.
 3. Model-generated paths MUST be validated before filesystem access.
 4. Local inference failure MUST NOT trigger Groq fallback.
 5. Groq mode MUST NEVER be presented as air-gapped.
 6. Groq API keys MUST never be committed or logged.
 7. Sandbox execution MUST have no network access.
 8. Generated code MUST NOT receive unrestricted host filesystem access.
 9. External egress in local mode MUST be denied.
 10. Retrieved evidence MUST retain provenance.
 11. Artifact claims based on retrieved evidence MUST be verification-capable.
 12. Agent execution MUST be bounded by retry and timeout limits.
 13. Unauthorized tools MUST fail closed and be audited.
 14. Resource exhaustion MUST result in controlled failure rather than uncontrolled model loading.
-

23. Security Test Matrix

Attack / Failure	Expected Result
Prompt injection inside uploaded PDF	Content is treated as untrusted data, not authority
<code>../../../../</code> path traversal	Policy validation rejects request
Absolute path outside workspace	Rejected
Unauthorized tool call	Rejected and audited
Invalid tool arguments	Rejected before execution
Sandbox attempts network access	Connection fails
Sandbox attempts host filesystem access	Access denied
Local mode attempts external endpoint	Blocked
Groq key missing	Clean provider error
Local provider unavailable	Fail, never switch to Groq
False citation	VERIFY fails and enters REPAIR
Invalid artifact structure	Artifact validation fails
Calculation mismatch	Verification/validation fails

Attack / Failure	Expected Result
Infinite agent loop	Hard task timeout
More than 3 repair attempts	Task fails safely
Tool timeout	Tool call fails and is audited
Excessive generated output	Resource limit triggers controlled failure
Malformed uploaded document	Ingestion fails safely
Invalid model registry entry	Model unavailable with clear error
Insufficient VRAM/RAM	Resource manager prevents unsafe model load

24. Development Hardware Profiles

RTX 3050A 4GB

Reasoning: Qwen2.5-3B Q4
Coding: Qwen2.5-Coder-1.5B or 3B Q4
Vision: No resident VLM; Tesseract CPU + Groq testing path if needed

Confirm available memory with:

```
nvidia-smi
```

Mac Apple Silicon

Reasoning: Qwen2.5-7B Q4
Coding: Qwen2.5-Coder-7B Q4
Vision: Qwen2.5-VL-3B Q4

7B vision deployment depends on available RAM and must be verified experimentally.

25. Five-Day Build Plan

Day 1 — Contracts + Skeleton

- Repository/module structure
- FastAPI

- Configuration
- Inference provider interface
- Ollama provider
- Groq provider
- Model registry
- Resource Manager skeleton
- Task state
- `/api/chat`
- SSE
- Frontend contract reconciliation

Day 2 — Agent Execution

- State machine
- Dynamic planning
- Tool schemas
- Policy Validation Layer
- Tool Registry
- bounded retries
- timeouts
- task persistence

Day 3 — RAG + Tools + Artifacts

- File upload
- Parsing/OCR interface
- Qdrant integration
- Retrieval + provenance
- Citation consistency verification
- DOCX generation
- Artifact manifests
- Artifact validation
- Coding sandbox
- Test/repair loop

Day 4 — Security + P&ID + Sovereignty

- Tool allow-lists

- Argument validation
- Sandbox restrictions
- Network isolation
- Sovereignty Monitor
- Audit events
- P&ID API
- Deterministic calculator
- Security test matrix

Day 5 — Integration + Failure Testing

Test:

- Local mode
- Groq mode
- Missing local model
- Provider failure
- No Groq fallback
- Unauthorized tool
- Path traversal
- Sandbox network attempt
- Malformed document
- Prompt injection
- Bad citation
- Failed calculation
- Failed code test
- Repair loop
- Timeout
- Resource exhaustion
- Human approval
- Artifact validation
- Audit completeness
- Sovereignty dashboard

Then freeze submission configuration:

```
INFERENCE_MODE=local
```

26. Acceptance Criteria

Inference

- ☐ Multiple models available
- ☐ Task-based model routing works
- ☐ Local hardware profile affects model selection
- ☐ Resource availability is checked
- ☐ New models can be added through registry configuration
- ☐ Local failure never silently switches to Groq

Agent

- ☐ Dynamic plan generated by model
- ☐ Multi-step execution works
- ☐ Tool calls are bounded
- ☐ Repair loop works
- ☐ Timeouts work
- ☐ Human approval works

Security

- ☐ Policy Validation Layer blocks unauthorized tools
- ☐ Tool arguments are validated
- ☐ Path traversal is blocked
- ☐ Sandbox has no network
- ☐ Local egress is denied
- ☐ Security events are audited
- ☐ Prompt injection does not directly gain tool authority

RAG / Evidence

- ☐ Documents are locally ingested
- ☐ Retrieval works
- ☐ Evidence has stable provenance
- ☐ Claims can be checked against cited chunks
- ☐ Failed citation verification triggers repair

Artifacts

- ☐ DOCX generation works
- ☐ Artifact opens correctly

- ☐ Artifact provenance is preserved
- ☐ Artifact hash is generated
- ☐ Artifact validation works
- ☐ Invalid artifacts are rejected

P&ID

- ☐ `/api/pid/analyze` works
- ☐ Backend receives normalized `PIDGraph`
- ☐ Confidence metadata is preserved

Sovereignty

- ☐ Local mode reports `AIR-GAPPED`
- ☐ Groq mode reports `EXTERNAL INFERENCE ACTIVE`
- ☐ Local external API calls remain zero
- ☐ Network policy is visible
- ☐ External byte counters are available
- ☐ Groq mode cannot be accidentally enabled as fallback

27. Core Demo Paths

Demo A — Inspection Report → Approval Note

```
Scanned inspection report
  ↓
OCR / visual extraction
  ↓
Local RAG
  ↓
Evidence retrieval
  ↓
Dynamic agent plan
  ↓
Verification
  ↓
DOCX generation
  ↓
Artifact validation
  ↓
Human approval
  ↓
Verified approval note
```

The resulting DOCX should contain, where applicable:

- inspection metadata
- extracted findings
- evidence references
- calculation steps
- assumptions
- recommendation
- approval fields
- generation timestamp
- artifact hash

Demo B — Coding

```
Coding request
  ↓
Coder model
  ↓
Policy validation
  ↓
Sandbox
  ↓
Tests
  ↓
Verification
  ↓
Repair if needed
  ↓
Verified code artifact
```

The code package should contain:

- generated source
- tests
- test results
- sandbox execution results
- resource-limit events
- verification report

Demo C — P&ID

```
P&ID image
  ↓
P&ID pipeline
  ↓
PIDGraph
  ↓
Confidence summary
  ↓
Artifact / narrative
  ↓
Audit
```

28. Final Engineering Principle

```
LLMs propose.
Policy decides.
Tools execute.
Verification checks.
Artifacts prove.
Audit records.
Network controls enforce sovereignty.
Humans approve consequential output.
```

Do not expand the architecture further during the five-day build unless a change is required to satisfy a demonstrated security, correctness, integration, or acceptance failure.